

Design Note: Assignment 1

1 Architecture & Pipeline

1.1 Reproducible Data Pipeline

To ensure full reproducibility, the pipeline was designed to execute via a single command (`python main.py build`). This script seamlessly manages the end-to-end process: downloading raw zip files directly from HuggingFace and AWS S3, extracting them, and parsing them into a unified, language-agnostic schema.

1.2 Custom Temporal Splitting & Future Leakage Prevention

To effectively capture user recency constraints, the interaction data is split strictly sequentially by timestamp rather than randomly. The pipeline dynamically calculates the maximum timestamp in the unified interaction logs and specifically reserves the **final 2 days** of the timeline strictly for the validation set.

Furthermore, a separate verification step independently asserts that no user history spans chronologically beyond the exact timestamp of an impression. This explicitly drops future clicks and prevents "future-click leakage", guaranteeing that the offline evaluation perfectly simulates a real-time production environment.

1.3 Alternatives Considered

- **Pandas vs. Polars for the Feature Store:** Initially, the pipeline utilized Pandas to group and join dense user-click arrays. However, memory overhead caused massive Out-Of-Memory (OOM) crashes on standard setups. We abandoned Pandas in favor of **Polars**. Its lazy evaluation engine and zero-copy string handling gracefully managed out-of-core processing.
- **Standard Mean-Pooling vs. Exponential Decay:** During early semantic experimentation on EB-NeRD, a standard mean-pooling of all historical user clicks severely diluted current user intent. We abandoned standard mean-pooling for EB-NeRD in favor of an **exponential time-decay algorithm**, giving dominant mathematical weight to recent clicks to adapt to highly volatile breaking news.

2 Retrieval Strategies

2.1 Lexical Candidate Generation (BM25)

The lexical retrieval system utilizes a heavily customized implementation of Okapi BM25. Instead of relying on slow, repeated linear scans, the pipeline pre-computes an inverted index mapping tokens directly to article IDs and term frequencies, guaranteeing $O(1)$ lookup times during candidate generation.

- **Query Formulation Engineering:** Rather than simply concatenating past article titles, we apply a sophisticated multi-signal approach for EB-NeRD. The query constructor extracts the top 15 highest-IDF distinctive terms from the user's recent history to eliminate noise.

Furthermore, it dynamically calculates and prepends the user’s dominant news category, providing immediate topical boosting.

- **Text Preprocessing:** For cross-lingual support, we apply language-specific tokenization, stop-word removal, and Snowball stemming (via NLTK) tailored for both English (MIND) and Danish (EB-NeRD).

2.2 Semantic Candidate Generation

The semantic retrieval pipeline captures contextual user intent that exact-keyword matching misses. Articles are embedded into dense vector spaces using HuggingFace sentence-transformers (all-MiniLM-L6-v2 for English, and paraphrase-multilingual-MiniLM-L12-v2 for Danish).

- **FAISS Indexing:** To accelerate nearest-neighbor retrieval across 100,000+ articles, we implement a FAISS IndexFlatIP structure, mathematically optimizing for Maximum Inner Product Search (equivalent to Cosine Similarity for normalized vectors).
- **User Representation:** User vectors are generated by pooling their click history. For MIND, we utilize standard mean-pooling. For EB-NeRD, we explicitly apply the aforementioned **exponential time-decay weight** to capture the rapidly changing, volatile nature of breaking Danish news, ensuring that recent clicks heavily dominate their semantic profile.
- **Hybrid Ensemble Scoring:** To test whether combining lexical and semantic signals would yield superior results, we implemented a late-fusion hybrid ranker. Because BM25 produces unbounded scores while Cosine Similarity operates in $[-1, 1]$, we applied Min-Max normalization to both score arrays per impression. A weighted ensemble was then computed using $Score = \alpha \cdot Lexical_{norm} + (1 - \alpha) \cdot Semantic_{norm}$. The hyperparameter α was tuned per dataset based on individual model strengths ($\alpha = 0.1$ for EB-NeRD; $\alpha = 0.5$ for MIND).

3 Evaluation & Observations

3.1 Consolidated Metrics Summary

Model	AUC (95% CI)	MRR	nDCG@10	Recall@50	Recall@200	ILD	Novelty
Semantic	0.5390 [0.532, 0.547]	0.3378	0.4542	0.0076	0.0350	0.7801	8.88
BM25	0.4942 [0.490, 0.498]	0.3073	0.4255	0.0071	0.0248	0.8433	8.93
Hybrid	0.5373 [0.530, 0.545]	0.3363	0.4527	0.0076	0.0350	0.7808	8.87

Table 1: Consolidated evaluation metrics for EB-NeRD.

Model	AUC (95% CI)	MRR	nDCG@10	Recall@50	Recall@200	ILD	Novelty
Semantic	0.4850 [0.483, 0.487]	0.2057	0.2695	0.0022	0.0046	0.8733	16.66
BM25	0.5335 [0.532, 0.535]	0.2464	0.3170	0.0063	0.0215	0.9431	16.66
Hybrid	0.5170 [0.515, 0.519]	0.2359	0.3040	0.0022	0.0046	0.9156	16.66

Table 2: Consolidated evaluation metrics for MIND.

3.2 The Dataset Inversion (Lexical vs. Semantic)

As shown in the summary table, a fascinating divergence in model performance was observed:

- **EB-NeRD (Danish):** Semantic embeddings completely dominated the lexical approach, achieving an AUC of **0.5390** and MRR of **0.3378**. The multilingual MiniLM model demonstrated a superior ability to capture contextual intent in Danish news.
- **MIND (English):** The trend inverted entirely. BM25 exact-matching crushed the semantic model, achieving an AUC of **0.5335** and MRR of **0.2464**. This suggests the base all-MiniLM-L6-v2 model struggles to differentiate the highly specific, dense vernacular of English news titles without domain-specific fine-tuning.
- **The Hybrid Dilution Effect:** Interestingly, the Hybrid ranker did not yield the expected "best of both worlds" ensemble boost. On EB-NeRD, despite heavily weighting the semantic score ($\alpha = 0.1$), the Hybrid AUC of 0.5373 slightly underperformed the pure Semantic model (0.5390). This reveals that for Danish news, BM25 exact-matching acts as destructive noise rather than a supplemental signal, effectively diluting the high-quality dense contextual vectors.

3.3 Anti-Gaming & Serving Constraints

We simulated strict real-time serving constraints by artificially dropping the **abstract** and **body** features during evaluation, forcing the models to rely solely on the article **title**.

- **Lexical Title-Density:** Surprisingly, stripping the abstracts *improved* (or perfectly maintained) the BM25 performance (e.g., EBNERD BM25 AUC increased from 0.4942 to 0.4987). This reveals that news abstracts often introduce "noisy" vocabulary that actively confuses exact-keyword matching, whereas the title provides a much cleaner, high-density signal for user intent.
- **Semantic Caching Quirk:** The semantic evaluation proved mathematically robust to the missing abstracts. Because semantic embeddings are computationally expensive, they are pre-encoded and cached in the feature store during the **build** phase. Dropping the text columns dynamically at evaluation time did not corrupt the pre-computed mathematical vectors, yielding identical performance metrics. To accurately measure the semantic model's strict title-only ablation performance in the future, a separate, distinct set of embeddings encoded exclusively from the title text would need to be generated and cached alongside the full-feature embeddings during the build phase.

3.4 Data Slicing: Cold-Start & Popularity

To analyze beyond-accuracy metrics, the dataset was sliced into user history quartiles and article popularity percentiles. Consistent trends emerged:

- **Cold-Start Users:** Short-history users generated notably higher MRR scores than Warm (long-history) users across all models (e.g., MIND BM25 Short-History MRR: 0.2698 vs. Long-History MRR: 0.2135). This strongly indicates that cold-start users are highly susceptible to clicking "head" (broadly popular/breaking) articles, which are statistically easier for unsupervised heuristics to retrieve.
- **Head vs. Tail Articles:** On EB-NeRD Semantic, Head articles (top 10% popular) achieved an MRR of 0.3915 versus 0.3553 for Tail articles (bottom 50% popular), confirming that semantic representations in breaking news gravitate towards high-density popularity clusters.

4 10x Scale Breakage

While the current architecture efficiently processes the standard scale (millions of logs), pushing this pipeline to 10x scale (e.g., 100 million+ impressions and 1.5 million articles) will expose two critical bottlenecks:

1. **FAISS IndexFlatIP Latency:** The semantic retrieval currently uses `IndexFlatIP`, which performs an exhaustive, exact L_2 distance calculation against every single article vector. At 10x scale, this exhaustive exact-search will cause catastrophic latency during batch candidate generation. The system must be refactored to use an approximated, quantized index such as `IndexIVFPQ` (Inverted File with Product Quantization) to trade a marginal drop in recall for sub-millisecond retrieval speeds.
2. **Dense Array Broadcasting Thrashing:** The vectorized BM25 candidate scoring algorithm relies on dense NumPy array broadcasting to process posting lists. While blazing fast at 100k articles, allocating massive dense matrices for matching token positions at 1.5M articles will cause severe memory thrashing and CPU cache misses. The scoring logic must be migrated to SciPy `csr_matrix` (Compressed Sparse Row) operations to handle the extreme sparsity of a 10x scale inverted index.
3. **Feature Store I/O Bottlenecks:** Currently, user interaction histories are loaded into memory-mapped Parquet files. At 10x scale (100M+ impressions), attempting to rapidly query recent click histories for millions of concurrent active users to generate query strings or mean-pooled vectors will create severe disk I/O latency. To survive this load, the feature store must be migrated from static Parquet files to a distributed, in-memory architecture. Implementing a Dynamo-style distributed hash table utilizing consistent hashing would allow us to safely shard the millions of user state profiles across a cluster of nodes, ensuring high availability and low-latency history lookups during candidate generation.